WEEK 10 – ADVANCED SQL QUERY SYSTEM

1. Introduction

The aim of this practical is to implement advanced SQL queries using subqueries and nested queries. These queries help retrieve and analyse data from multiple tables in a database.

For the ArtHub project, advanced SQL techniques are used to analyse artwork prices, customer purchases, product sales, category performance, and business trends. This analysis helps identify valuable customers, expensive artworks, best-selling products, and business performance.

2. Objectives

- To understand and implement subqueries and nested queries.
- To identify artworks priced above the average price.
- To analyse customer purchases and spending patterns.
- To identify the best-selling and high-revenue artworks.
- To perform category-wise sales analysis.
- To generate advanced business reports for decision-making.

3. Implementation of Subqueries and Nested Queries

A subquery is a query written inside another SQL query. It is used to retrieve data that is required by the main query.

Subqueries help perform complex database operations by dividing them into smaller steps.

Types of Subqueries

1. Single-Row Subquery

A single-row subquery returns only one row as its result. It is commonly used with comparison operators such as greater than, less than, and equal to.

2. Multi-Row Subquery

A multi-row subquery returns more than one row. It is used with operators such as IN, ANY, and ALL.

3. Nested Subquery

A nested subquery contains one or more subqueries inside another query. It helps perform complex data retrieval and analysis.

Application in ArtHub

Subqueries are implemented to calculate the average artwork price and identify artworks whose prices are higher than the average price. This helps identify premium and expensive artworks in the ArtHub database.

4. Finding Products Above Average Price

This section analyses artwork prices stored in the ArtProduct table.

The following operations are performed:

- Identifying artworks whose prices are greater than the overall average price.

```
SQL> SELECT AVG(PRICE) AS AVERAGE_PRICE FROM ArtProduct;

AVERAGE_PRICE
-------------
         3870
```

```
SQL> SELECT ARTWORK_NAME, PRICE FROM ArtProduct WHERE PRICE > (SELECT AVG(PRICE) FROM ArtProduct);

ARTWORK_NAME
--------------------------------------------------------------------------------
     PRICE
----------
Sunset Over Hills
      5000

Colors of Nature
      5200

Future City
      4000


ARTWORK_NAME
--------------------------------------------------------------------------------
     PRICE
----------
The Thinker Sculpture
      7500

Abstract Dreams
      4200
```

- Displaying artwork name, category ID, and price.

```
SQL> SELECT ARTWORK_NAME, CATEGORY_ID, PRICE FROM ArtProduct WHERE PRICE > (SELECT AVG(PRICE) FROM ArtProduct);

ARTWORK_NAME
--------------------------------------------------------------------------------
CATEGORY_I      PRICE
---------- ----------
Sunset Over Hills
1               5000

Colors of Nature
1               5200

Future City
3               4000


ARTWORK_NAME
--------------------------------------------------------------------------------
CATEGORY_I      PRICE
---------- ----------
The Thinker Sculpture
5               7500

Abstract Dreams
9               4200
```

- Identifying expensive artworks within each category.

```
SQL> SELECT A1.ARTWORK_NAME, A1.CATEGORY_ID, A1.PRICE FROM ArtProduct A1 WHERE A1.PRICE > (SELECT AVG(A2.PRICE) FROM ArtProduct A2 WHERE A2.CATEGORY_ID = A1
.CATEGORY_ID);

ARTWORK_NAME
--------------------------------------------------------------------------------
CATEGORY_I      PRICE
---------- ----------
Colors of Nature
1              5200

Charcoal Dreams
2              2200

Future City
3              4000
```

5. Identifying Customers with Maximum Purchases

Customer purchase analysis is performed using the Customer and Orders tables.

The following operations are carried out:

- Identifying the customer who spent the highest amount.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM Customer C JOIN Orders O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.
CUSTOMER_ID, C.CUSTOMER_NAME HAVING SUM(O.TOTAL_AMOUNT) = (SELECT MAX(TOTAL_SPENDING) FROM (SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING FROM Orders GROUP BY
CUSTOMER_ID));

CUSTOMER_ID CUSTOMER_NAME                  TOTAL_SPENDING
----------- ------------------------------ --------------
          2 Virat                                    5200
```

- Finding customers with the maximum number of orders.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS TOTAL_ORDERS FROM Customer C JOIN Orders O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUST
OMER_ID, C.CUSTOMER_NAME HAVING COUNT(O.ORDER_ID) = (SELECT MAX(ORDER_COUNT) FROM (SELECT COUNT(*) AS ORDER_COUNT FROM Orders GROUP BY CUSTOMER_ID));

CUSTOMER_ID CUSTOMER_NAME                  TOTAL_ORDERS
----------- ------------------------------ ------------
          4 SanjuSamson                               1
          1 Dhoni                                     1
          6 Hardhik Pandya                            1
          3 Ashwin                                    1
          5 Shreyas Iyer                              1
          2 Virat                                     1
          7 Ruturaj Gaikwad                           1

7 rows selected.
```

- Identifying customers whose spending is above the average customer spending.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM Customer C JOIN Orders O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.
CUSTOMER_ID, C.CUSTOMER_NAME HAVING SUM(O.TOTAL_AMOUNT) > (SELECT AVG(CUSTOMER_SPENDING) FROM (SELECT SUM(TOTAL_AMOUNT) AS CUSTOMER_SPENDING FROM Orders GRO
UP BY CUSTOMER_ID));

CUSTOMER_ID CUSTOMER_NAME                  TOTAL_SPENDING
----------- ------------------------------ --------------
          1 Dhoni                                    5000
          6 Hardhik Pandya                           4000
          2 Virat                                    5200
```

- Displaying the top five valuable customers based on total spending.

```
SQL> SELECT * FROM (SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM Customer C JOIN Orders O ON C.CUSTOMER_ID = O.CUSTOMER
_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME ORDER BY TOTAL_SPENDING DESC) WHERE ROWNUM <= 5;

CUSTOMER_ID CUSTOMER_NAME                  TOTAL_SPENDING
----------- ------------------------------ --------------
          2 Virat                                    5200
          1 Dhoni                                    5000
          6 Hardhik Pandya                           4000
          5 Shreyas Iyer                             3500
          7 Ruturaj Gaikwad                          2800
```

This analysis helps understand customer purchasing behaviour and identify customers who contribute significantly to the business.

6. Generating Complex Business Queries

Complex business queries are used to analyse product performance, customer value, category performance, and purchase history.

6.1 Best-Selling Product

The best-selling artwork is identified based on the total quantity sold.

The report displays:

- Artwork name.
- Total quantity sold.
- Total revenue generated.

```
SQL> SELECT A.ARTWORK_NAME, SUM(D.QUANTITY) AS TOTAL_QUANTITY_SOLD, SUM(D.QUANTITY * D.PRICE) AS TOTAL_REVENUE FROM ArtProduct A JOIN OrderDetails D ON A.AR
TWORK_ID = D.ARTWORK_ID GROUP BY A.ARTWORK_NAME HAVING SUM(D.QUANTITY) = (SELECT MAX(TOTAL_QUANTITY) FROM (SELECT SUM(QUANTITY) AS TOTAL_QUANTITY FROM Order
Details GROUP BY ARTWORK_ID));

ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
Sunset Over Hills
                  1          5000

Pencil Portrait
                  1          1800

Colors of Nature
                  1          5200


ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
Digital Galaxy
                  1          3500

Charcoal Dreams
                  1          2200

Future City
                  1          4000


6 rows selected.
```

This helps identify the artworks that are most popular among customers.

6.2 High-Value Customers

Customers whose total purchase amount is greater than the average customer spending are identified.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM Customer C JOIN Orders O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.
CUSTOMER_ID, C.CUSTOMER_NAME HAVING SUM(O.TOTAL_AMOUNT) > (SELECT AVG(CUSTOMER_SPENDING) FROM (SELECT SUM(TOTAL_AMOUNT) AS CUSTOMER_SPENDING FROM Orders GRO
UP BY CUSTOMER_ID));

CUSTOMER_ID CUSTOMER_NAME                  TOTAL_SPENDING
----------- ------------------------------ --------------
          1 Dhoni                                    5000
          6 Hardhik Pandya                           4000
          2 Virat                                    5200
```

This analysis helps the business recognise valuable customers and understand their contribution to total sales.

6.3 Category Performance Analysis

Category-wise performance is analysed to understand the sales contribution of different artwork categories.

The analysis includes:

- Identifying the highest revenue-generating category.

```
SQL> SELECT A.CATEGORY_ID, SUM(D.QUANTITY * D.PRICE) AS TOTAL_REVENUE FROM ArtProduct A JOIN OrderDetails D ON A.ARTWORK_ID = D.ARTWORK_ID GROUP BY A.CATEGO
RY_ID HAVING SUM(D.QUANTITY * D.PRICE) = (SELECT MAX(CATEGORY_REVENUE) FROM (SELECT SUM(D2.QUANTITY * D2.PRICE) AS CATEGORY_REVENUE FROM ArtProduct A2 JOIN
OrderDetails D2 ON A2.ARTWORK_ID = D2.ARTWORK_ID GROUP BY A2.CATEGORY_ID));

CATEGORY_I TOTAL_REVENUE
---------- -------------
1                 10200
```

- Calculating the number of products sold in each category.

```
SQL> SELECT A.CATEGORY_ID, SUM(D.QUANTITY) AS TOTAL_PRODUCTS_SOLD FROM ArtProduct A JOIN OrderDetails D ON A.ARTWORK_ID = D.ARTWORK_ID GROUP BY A.CATEGORY_I
D;

CATEGORY_I TOTAL_PRODUCTS_SOLD
---------- -------------------
1                            2
3                            2
2                            2
```

- Finding the average artwork price in each category.

```
SQL> SELECT CATEGORY_ID, AVG(PRICE) AS AVERAGE_PRICE FROM ArtProduct GROUP BY CATEGORY_ID;

CATEGORY_I AVERAGE_PRICE
---------- --------------
1                    5100
3                    3750
10                   2500
5                    7500
9                    4200
2                    2000
4                    2800

7 rows selected.
```

This helps compare category performance and identify categories that generate higher revenue.

6.4 Customer Purchase History

Customer purchase history is analysed to understand individual purchasing patterns.

The report displays:

- Customer name.
- Number of orders.
- Total purchase amount.
- Most purchased product.

```
SQL> WITH PRODUCT_COUNT AS (SELECT O.CUSTOMER_ID, A.ARTWORK_NAME, SUM(D.QUANTITY) AS TOTAL_QTY, ROW_NUMBER() OVER (PARTITION BY O.CUSTOMER_ID ORDER BY SUM(D
.QUANTITY) DESC) AS RN FROM Orders O JOIN OrderDetails D ON O.ORDER_ID = D.ORDER_ID JOIN ArtProduct A ON D.ARTWORK_ID = A.ARTWORK_ID GROUP BY O.CUSTOMER_ID,
 A.ARTWORK_NAME), CUSTOMER_TOTAL AS (SELECT CUSTOMER_ID, COUNT(*) AS NUMBER_OF_ORDERS, SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE_AMOUNT FROM Orders GROUP BY CUSTO
MER_ID) SELECT C.CUSTOMER_NAME, NVL(T.NUMBER_OF_ORDERS,0) AS NUMBER_OF_ORDERS, NVL(T.TOTAL_PURCHASE_AMOUNT,0) AS TOTAL_PURCHASE_AMOUNT, P.ARTWORK_NAME AS MO
ST_PURCHASED_PRODUCT FROM Customer C LEFT JOIN CUSTOMER_TOTAL T ON C.CUSTOMER_ID = T.CUSTOMER_ID LEFT JOIN PRODUCT_COUNT P ON C.CUSTOMER_ID = P.CUSTOMER_ID
AND P.RN = 1;

CUSTOMER_NAME                  NUMBER_OF_ORDERS TOTAL_PURCHASE_AMOUNT
------------------------------ ---------------- ---------------------
MOST_PURCHASED_PRODUCT
--------------------------------------------------------------------------------
Dhoni                                         1                  5000
Sunset Over Hills

Hardhik Pandya                                1                  4000
Future City

Virat                                         1                  5200
Colors of Nature


CUSTOMER_NAME                  NUMBER_OF_ORDERS TOTAL_PURCHASE_AMOUNT
------------------------------ ---------------- ---------------------
MOST_PURCHASED_PRODUCT
--------------------------------------------------------------------------------
SanjuSamson                                   1                  2200
Charcoal Dreams

Shreyas Iyer                                  1                  3500
Digital Galaxy

Ashwin                                        1                  1800
Pencil Portrait


CUSTOMER_NAME                  NUMBER_OF_ORDERS TOTAL_PURCHASE_AMOUNT
------------------------------ ---------------- ---------------------
MOST_PURCHASED_PRODUCT
--------------------------------------------------------------------------------
Ruturaj Gaikwad                               1                  2800
```

This information helps understand customer interests and purchasing frequency.

7. Preparing Advanced SQL Reports

Advanced SQL reports are generated to support business analysis and decision-making in the ArtHub system.

7.1 Premium Product Report

The Premium Product Report identifies artworks whose prices are above the average artwork price.

The report includes:

- Artwork name.
- Category ID.
- Artwork price.
- Stock availability condition.

```
SQL> SELECT ARTWORK_NAME, CATEGORY_ID, PRICE, CASE WHEN STOCK > 0 THEN 'IN STOCK' ELSE 'OUT OF STOCK' END AS STOCK_AVAILABILITY FROM ArtProduct WHERE PRICE
> (SELECT AVG(PRICE) FROM ArtProduct);

ARTWORK_NAME
--------------------------------------------------------------------------------
CATEGORY_I      PRICE STOCK_AVAILA
---------- ---------- ------------
Sunset Over Hills
1                5000 IN STOCK

Colors of Nature
1                5200 IN STOCK

Future City
3                4000 IN STOCK


ARTWORK_NAME
--------------------------------------------------------------------------------
CATEGORY_I      PRICE STOCK_AVAILA
---------- ---------- ------------
The Thinker Sculpture
5                7500 IN STOCK

Abstract Dreams
9                4200 IN STOCK
```

Stock availability is classified as either IN STOCK or OUT OF STOCK based on the available quantity.

This report helps identify expensive artworks and monitor their availability.

7.2 Customer Value Report

The Customer Value Report analyses the purchasing value of each customer.

The report includes:

- Customer name.
- Total number of orders.
- Total spending.
- Customer category.

Customers spending ₹4,000 or more are classified as PREMIUM, while customers spending below ₹4,000 are classified as REGULAR.

```
SQL> SELECT C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS TOTAL_ORDERS, NVL(SUM(O.TOTAL_AMOUNT),0) AS TOTAL_SPENDING, CASE WHEN NVL(SUM(O.TOTAL_AMOUNT),0) >= 4000 T
HEN 'PREMIUM' ELSE 'REGULAR' END AS CUSTOMER_CATEGORY FROM Customer C LEFT JOIN Orders O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER
_NAME;

CUSTOMER_NAME                  TOTAL_ORDERS TOTAL_SPENDING CUSTOME
------------------------------ ------------ -------------- -------
SanjuSamson                               1           2200 REGULAR
Jadeja                                    0              0 REGULAR
Rohit Sharma                              0              0 REGULAR
Dhoni                                     1           5000 PREMIUM
Hardhik Pandya                            1           4000 PREMIUM
Ashwin                                    1           1800 REGULAR
Shreyas Iyer                              1           3500 REGULAR
Virat                                     1           5200 PREMIUM
Ruturaj Gaikwad                           1           2800 REGULAR
Raina                                     0              0 REGULAR

10 rows selected.
```

This classification helps identify valuable customers and understand customer contribution to the business.

7.3 Sales Performance Report

The Sales Performance Report evaluates the sales performance of artworks and their categories.

The report includes:

Product-wise Sales

- Artwork name.
- Total quantity sold.
- Total revenue generated.

```
SQL> SELECT A.ARTWORK_NAME, NVL(SUM(D.QUANTITY),0) AS TOTAL_QUANTITY_SOLD, NVL(SUM(D.QUANTITY * D.PRICE),0) AS TOTAL_REVENUE FROM ArtProduct A LEFT JOIN Ord
erDetails D ON A.ARTWORK_ID = D.ARTWORK_ID GROUP BY A.ARTWORK_ID, A.ARTWORK_NAME;

ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
Handmade Craft
                  0             0

Charcoal Dreams
                  1          2200

Colors of Nature
                  1          5200


ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
The Thinker Sculpture
                  0             0

Future City
                  1          4000

Sunset Over Hills
                  1          5000


ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
Pencil Portrait
                  1          1800

Digital Galaxy
                  1          3500
```

Category-wise Revenue

- Category ID.
- Total revenue generated by each category.

```
SQL> SELECT A.CATEGORY_ID, NVL(SUM(D.QUANTITY * D.PRICE),0) AS TOTAL_REVENUE FROM ArtProduct A LEFT JOIN OrderDetails D ON A.ARTWORK_ID = D.ARTWORK_ID GROUP
 BY A.CATEGORY_ID;

CATEGORY_I TOTAL_REVENUE
---------- -------------
1                 10200
3                  7500
10                    0
5                     0
9                     0
2                  4000
4                     0

7 rows selected.
```

Top-Selling Products

- Identifying artworks with the highest quantity sold.
- Displaying their total quantity and revenue.

```
SQL> SELECT A.ARTWORK_NAME, SUM(D.QUANTITY) AS TOTAL_QUANTITY_SOLD, SUM(D.QUANTITY * D.PRICE) AS TOTAL_REVENUE FROM ArtProduct A JOIN OrderDetails D ON A.AR
TWORK_ID = D.ARTWORK_ID GROUP BY A.ARTWORK_ID, A.ARTWORK_NAME HAVING SUM(D.QUANTITY) = (SELECT MAX(TOTAL_QUANTITY) FROM (SELECT SUM(QUANTITY) AS TOTAL_QUANT
ITY FROM OrderDetails GROUP BY ARTWORK_ID));

ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
Charcoal Dreams
                  1          2200

Colors of Nature
                  1          5200

Future City
                  1          4000


ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
Sunset Over Hills
                  1          5000

Pencil Portrait
                  1          1800

Digital Galaxy
                  1          3500


6 rows selected.
```

This report helps analyse product popularity and overall sales performance.

7.4 Business Decision Report

The Business Decision Report provides useful information for making business decisions.

The following analyses are performed:

High-Performing Products: Identifies artworks generating the highest sales revenue.

```
SQL> SELECT A.ARTWORK_NAME, SUM(D.QUANTITY) AS TOTAL_QUANTITY_SOLD, SUM(D.QUANTITY * D.PRICE) AS TOTAL_REVENUE FROM ArtProduct A JOIN OrderDetails D ON A.AR
TWORK_ID = D.ARTWORK_ID GROUP BY A.ARTWORK_ID, A.ARTWORK_NAME HAVING SUM(D.QUANTITY * D.PRICE) = (SELECT MAX(PRODUCT_REVENUE) FROM (SELECT SUM(QUANTITY * PR
ICE) AS PRODUCT_REVENUE FROM OrderDetails GROUP BY ARTWORK_ID));

ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
Colors of Nature
                  1          5200
```

Low-Performing Products: Identifies artworks with zero or one unit sold. These products may require additional promotion.

```
SQL> SELECT A.ARTWORK_NAME, NVL(SUM(D.QUANTITY),0) AS TOTAL_QUANTITY_SOLD FROM ArtProduct A LEFT JOIN OrderDetails D ON A.ARTWORK_ID = D.ARTWORK_ID GROUP BY
 A.ARTWORK_ID, A.ARTWORK_NAME HAVING NVL(SUM(D.QUANTITY),0) <= 1;

ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
Handmade Craft
                  0

Charcoal Dreams
                  1

Colors of Nature
                  1


ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
The Thinker Sculpture
                  0

Future City
                  1

Sunset Over Hills
                  1


ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
Pencil Portrait
                  1

Digital Galaxy
                  1
```

High-Value Customers: Identifies customers whose total spending is ₹4,000 or more.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM Customer C JOIN Orders O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.
CUSTOMER_ID, C.CUSTOMER_NAME HAVING SUM(O.TOTAL_AMOUNT) >= 4000;

CUSTOMER_ID CUSTOMER_NAME                  TOTAL_SPENDING
----------- ------------------------------ --------------
          1 Dhoni                                    5000
          6 Hardhik Pandya                           4000
          2 Virat                                    5200
```

Revenue Trends: Analyses the total number of orders and revenue generated on each order date.

```
SQL> SELECT TRUNC(ORDER_DATE) AS ORDER_DATE, COUNT(ORDER_ID) AS TOTAL_ORDERS, SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE FROM Orders GROUP BY TRUNC(ORDER_DATE) ORDE
R BY ORDER_DATE;

ORDER_DATE         TOTAL_ORDERS TOTAL_REVENUE
------------------ ------------ -------------
20-SEP-26                     1          5000
21-SEP-26                     2          7000
22-SEP-26                     2          5700
23-SEP-26                     1          4000
24-SEP-26                     1          2800
```

These reports help the business understand sales trends, improve product promotion, and identify valuable customers.

8. Result

Thus, the Week 10 practical on Advanced SQL Query System was successfully completed using the ArtHub database. Subqueries and nested queries were implemented to analyse artwork prices, customer purchases, product sales, category performance, and business trends.

The advanced SQL reports helped identify premium products, high-value customers, best-selling artworks, low-performing products, and revenue trends. This practical improved the understanding of advanced SQL concepts and their applications in real-world business analysis.

9. Conclusion

The Week 10 practical provided knowledge about the implementation of subqueries and nested queries in Oracle SQL. Different advanced queries were used to retrieve and analyse information from the ArtHub database.

The generated reports provide useful insights into customer behaviour, artwork sales, category performance, and business revenue. These insights can support better business decisions and improve the overall management of the ArtHub system.